

TRM

An Efficient Hypervisor-Based Framework for Malware Analysis and Memory Reconstruction

Mohammad Sina Karvandi · Soroush Meghdadizanjani · Sima Arasteh
Saleh Khalaj Monfared · Mohammad K. Fallah · Saeid Gorgin
Jeong-A Lee · Asia Slowinska · Erik van der Kouwe

ACM ASIA CCS 2026 · Bangalore, India · June 1–5, 2026



We automatically reconstructed BlackLotus's internal structures.

No source code. No debug symbols. The malware never detected us.

In this talk: how we built TRM - the tool that made it possible.

43%

faster
reverse engineering

+57%

overhead only
(vs. +142% prior)



Better stealthiness
to malware

Motivation: The Rootkit Analysis Gap

Modern Rootkits

Exploit kernel-mode drivers, bypass KPP & DSE, evade AV/EDR using kernel privileges

Dynamic Tools Are Slow

62% of analysts say sandbox tools are too slow; instrumentation incurs high overhead

Static Analysis Fails

Packing, obfuscation, and compilation information loss render static tools ineffective

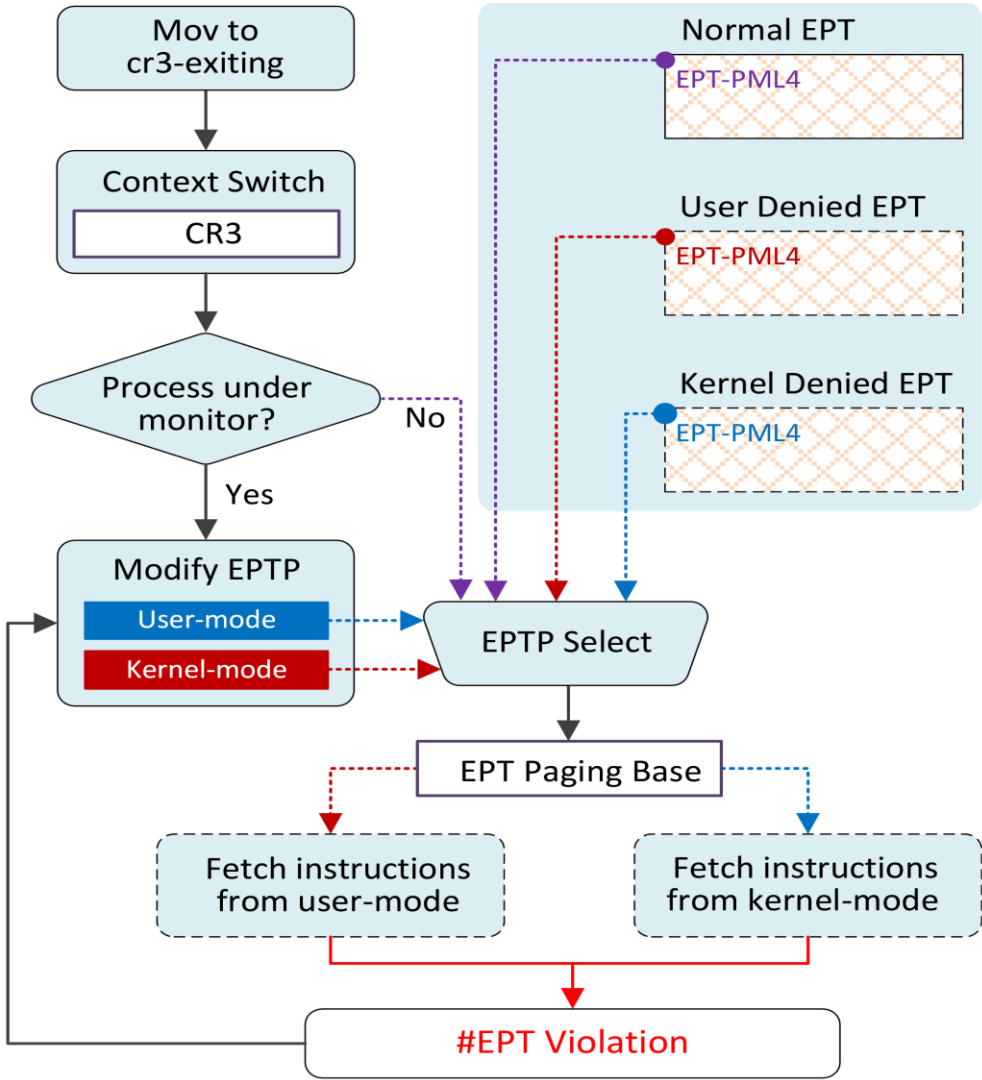
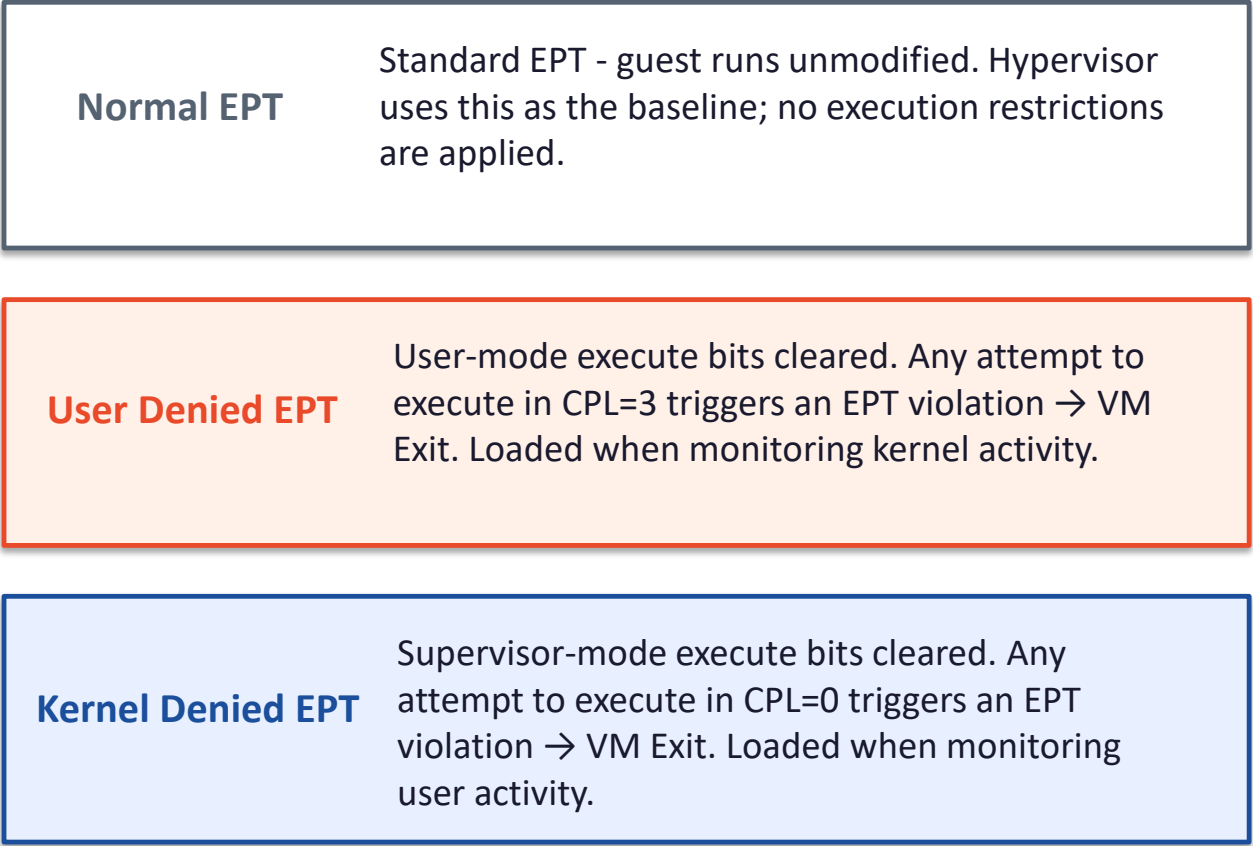
Blind Spots

Intel PIN, Valgrind, DynamoRIO have no kernel-mode visibility; QEMU is too slow for real-world use

Key Question: Can structure reconstruction be performed in hostile kernel environments without prohibitive cost?

Contribution 1: Execution Mode Detection via MBEC

Novel primitive for user ↔ kernel transition monitoring



Use Case: BlackLotus UEFI Bootkit

Real-world stealthy rootkit analysis

BlackLotus Profile

- UEFI bootkit - bypasses Secure Boot & BitLocker
- Disables HVCI, UAC, Windows Defender, EDRs
- Anti-kernel-debug, anti-hypervisor, anti-sandbox
- Source code partially leaked (verified findings)

Anti-Debug Bypass

TRM uses no OS debug APIs →
bypasses BeingDebugged PEB
flag and all kernel-debugger
checks (KD present flag)

C&C Structure Reconst.

Reconstructed full structure of
HTTP downloader ↔ C&C
communication buffer (see
Table 3 in paper)

And lots of other discoveries (check the paper).

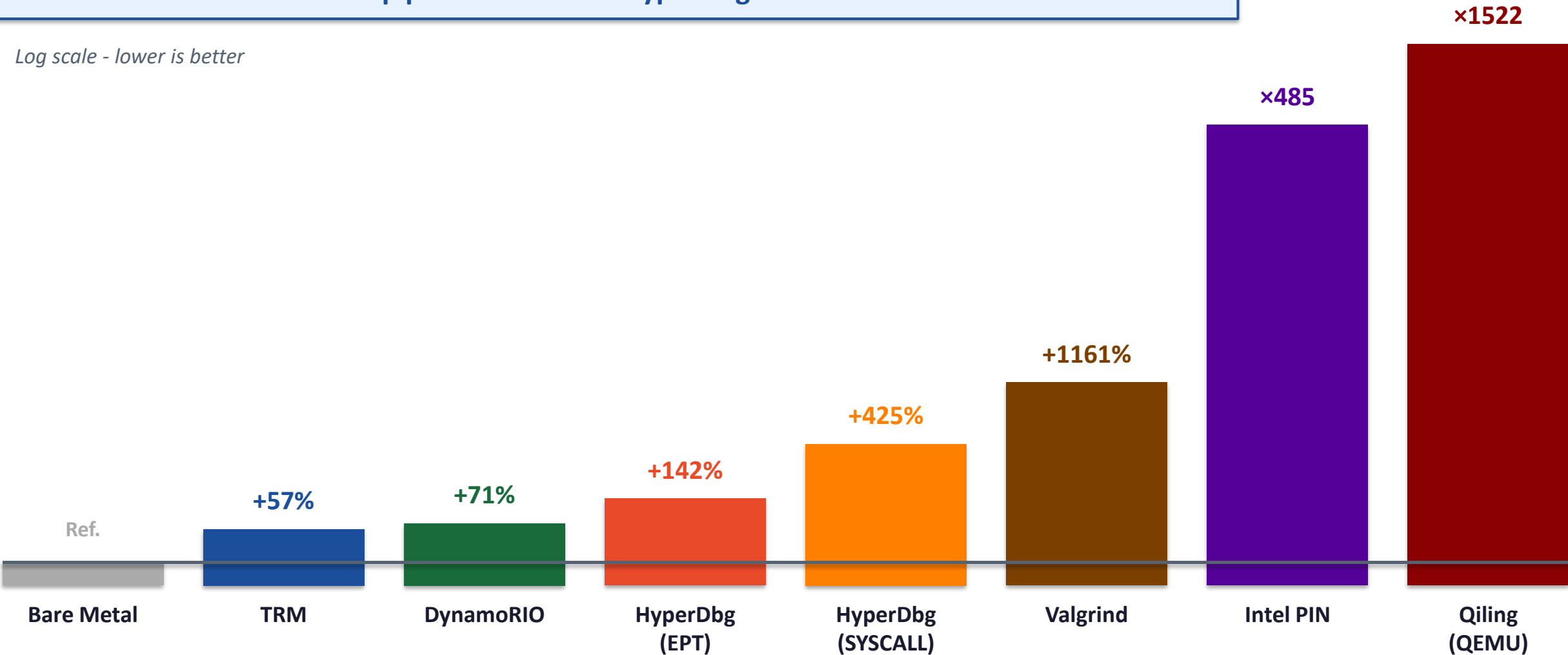
Key insight: TRM's transparency layer (via HyperDbg) bypasses ALL BlackLotus anti-analysis checks - the malware never detects it is being analyzed

Evaluation: Overhead Comparison

User-to-kernel transition monitoring — TRM vs. prior tools

TRM: +57% overhead · DynamoRIO: +71% · HyperDbg: +142% | TRM uses one hardware trap per transition — HyperDbg needs two

Log scale - lower is better



Evaluation: Structure Reconstruction Accuracy

Windows kernel structures & rootkit structures

Windows Kernel/User Structures

Structure	Un%	Acc%	Ms%
_OBJECT_HEADER	0%	100%	0%
_LIST_ENTRY	0%	100%	0%
_LARGE_INTEGER	0%	100%	0%
_PROCESS_BASIC_INFO	0%	100%	0%
_KAPC	17%	83%	0%
_POOL_HEADER	20%	80%	0%
_KTRAP_FRAME	26%	74%	0%
_EPROCESS	71%	29%	0%
tagHW_PROFILE_INFOA	0%	33%	67%

Rootkit Structures

Rootkit	Un%	Acc%	Ms%
KMDF5Driver Keylogger	53.3%	46.7%	6.7%
KMDF4Driver DKOM	40.0%	53.3%	6.7%
KMDF3Driver Subscribe	20.0%	80.0%	0.0%
Win_Rootkit	26.7%	60.0%	13.3%
Spectre Rootkit	56.6%	39.7%	3.7%
BlackLotus	54.5%	40.9%	4.6%
DrvHide	61.8%	38.2%	0.0%
Chaos-Rootkit	33.3%	53.3%	13.3%

Un% = fields never accessed at runtime | Acc% = correctly reconstructed | Ms% = misclassified (mainly consecutive arrays)

Conclusion

TRM - The Reversing Machine

Novel MBEC Primitive

First use of Mode-Based Execution Control for efficient user $\leftrightarrow$ kernel transition detection - single VM Exit per interception

EPT Memory Tracing

Hypervisor-assisted selective memory interception for function parameters and data structure field recovery

57% Overhead

Reduces user-kernel monitoring cost from 142% (HyperDbg) to 57% - practical for interactive malware analysis

Transparent Execution

Analyze binaries from first instruction without OS debug APIs - entirely below OS level, invisible to malware

Structure Reconstruction

First hypervisor-level automated runtime reconstruction - works for both user-mode and kernel-mode structures

43% Faster RE

User study: 43% speedup in reverse engineering time with improved accuracy over pure manual analysis

Open-source · github.com/HyperDbg/HyperDbg · github.com/HyperDbg/structure-reconstructor

Thank you! Questions welcome (m.s.karvandi@vu.nl)